MAGÍSTER EN DATA SCIENCE
Universidad del Desarrollo

Big Data y Cloud Computing

PROYECTO INTEGRADOR
FASE 3: DOCUMENTACIÓN TÉCNICA

Gestión Territorial Inteligente de Solicitudes Ciudadanas
Municipalidad de El Monte

Campo	Detalle
Curso	Big Data y Cloud Computing
Proyecto	Gestión Territorial Inteligente de Solicitudes Ciudadanas
Institución de referencia	Municipalidad de El Monte
Equipo	Grupo 2: Sofía Alanís, Osvaldo Neira, Jaime Sandoval, Dante Aguirre
Versión	Documento formal Fase 3
Base técnica	Repositorio generado por Codex: fase3_gestion_territorial_el_monte
Fecha	Junio de 2026

1. CONTEXTO DEL PROBLEMA Y OBJETIVO DE FASE 3 3

2. ARQUITECTURA FINAL IMPLEMENTADA 3

3. FLUJO DE DATOS BRONZE, SILVER Y GOLD 5

4. EVIDENCIA EJECUTADA Y RESULTADOS MEDIBLES 5

5. OPTIMIZACION, BENCHMARK Y LECTURA CRITICA 8

6. CUMPLIMIENTO FULL Y LIMITACIONES ACTUALES 9

7. MANUAL OPERATIVO RESUMIDO 10

8. GOBIERNO, SEGURIDAD, LINAJE Y FINOPS 10

9. PROPUESTA DE EVOLUCION GCP 11

10. CONCLUSIONES 11

DECLARACIÓN DE USO DE IA GENERATIVA. 11

ANEXO A - ARCHITECTURE DECISION RECORDS 12

ADR-001: ARQUITECTURA LAKEHOUSE MEDALLION 12
ADR-002: PYSPARK Y PARQUET PARA PROCESAMIENTO BATCH 12
ADR-003: EJECUCION REPRODUCIBLE EN COLAB CON RUTA A GCP ADMINISTRADO 13
ADR-004: BENCHMARK DE PARTICIONAMIENTO Y LECTURA CRITICA 13
ADR-005: GOBIERNO, SEGURIDAD Y DATOS SINTETICOS 14

1. Contexto del problema y objetivo de Fase 3

El proyecto aborda la gestion territorial de solicitudes ciudadanas de la Municipalidad de El Monte. El stakeholder principal es el equipo municipal que debe priorizar reclamos, coordinar cuadrillas, monitorear SLA, identificar zonas criticas y comunicar resultados a supervision y autoridad. El problema califica como Big Data porque no se limita a una tabla transaccional pequena: combina solicitudes, eventos de ciclo de vida, logs de plataforma, metadatos de fotografias, clima, cuadrillas, satisfaccion y datasets derivados. El perfil full definido para la arquitectura considera 50.000 solicitudes, 10.800.000 eventos y 1.500.000 logs, superando el umbral de 10 millones de filas en Bronze recomendado por el enunciado.

La Fase 3 documenta el sistema para que un Data Engineer que no participo en el desarrollo pueda entender la arquitectura, reproducir la ejecucion, operar el pipeline, interpretar la evidencia y evolucionar la solucion hacia un entorno administrado. En esta version, la evidencia adjunta demuestra una corrida end-to-end en `SCALE_MODE=full`, con mas de 12 millones de registros Bronze y estado final exitoso. El criterio de honestidad tecnica se mantiene: no se ajusto la escala full para hacerla mas facil, y los resultados del benchmark se reportan aunque la optimizacion medida no haya mejorado el tiempo de lectura.

Desde el punto de vista municipal, el valor no esta solamente en contar solicitudes, sino en transformar eventos dispersos en decisiones operacionales. Una direccion de operaciones necesita saber que territorios acumulan backlog, que tipos de solicitud rompen SLA, que cuadrillas muestran mejor tiempo de cierre, donde aparecen reaperturas y que zonas concentran baja satisfaccion. Por eso la arquitectura prioriza trazabilidad y reproducibilidad: cada KPI Gold debe poder volver a una solicitud, un evento o una regla de negocio en Silver, evitando tableros que funcionen como cajas negras.

2. Arquitectura final implementada

La arquitectura final mantiene el patron lakehouse medallion. Bronze conserva fuentes sinteticas crudas en Parquet; Silver aplica deduplicacion, validaciones, enriquecimiento y calculo de indicadores operacionales; Gold publica salidas analiticas para dashboard, BigQuery o Looker Studio. El procesamiento es batch con PySpark, porque el caso se orienta a consolidacion periodica de solicitudes y eventos, no a respuesta subsegundo. La solucion queda preparada para migrar desde Colab/local a GCP usando Cloud Storage como data lake, Dataproc o Spark Serverless como motor de procesamiento, BigQuery como capa analitica y Looker Studio como capa de visualizacion.

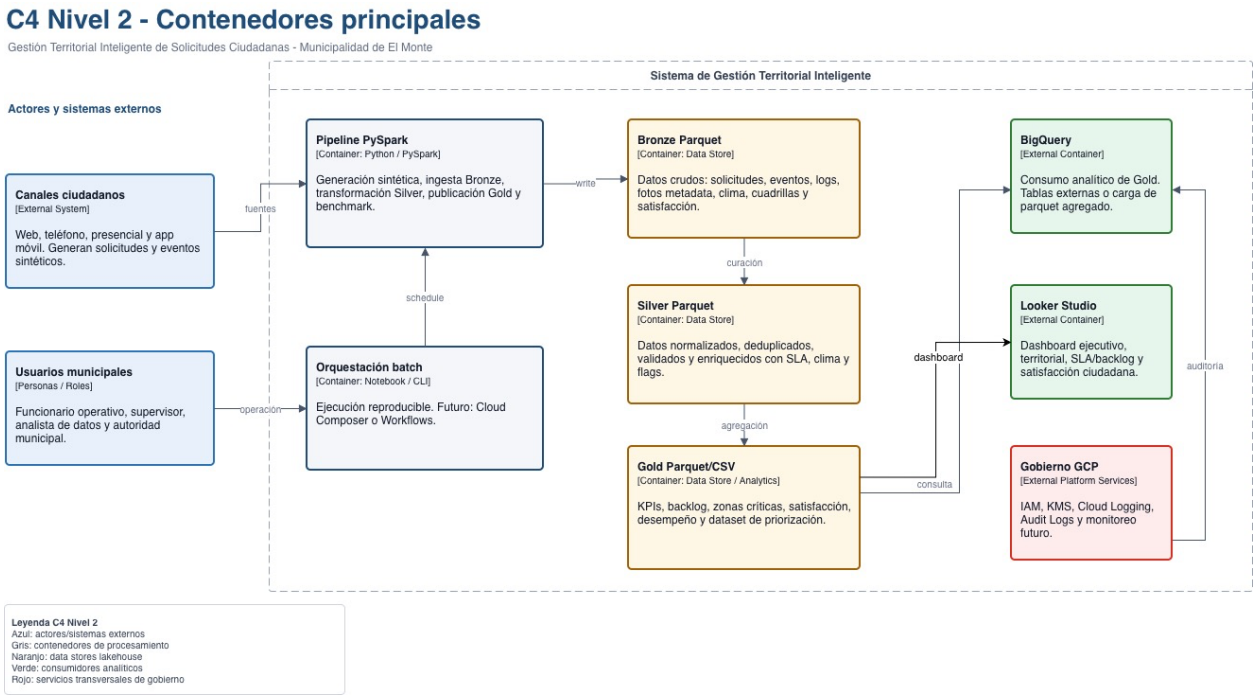


Figura 1. Diagrama C4 Nivel 2 de la arquitectura final.

El contenedor central es el pipeline PySpark, ejecutable por notebook o CLI. Este pipeline genera datos sinteticos con `spark.range`, escribe Bronze mediante una funcion segura con staging temporal, transforma Silver, publica Gold y registra evidencia. La separacion de contenedores facilita reemplazar la ejecucion local por Dataproc sin cambiar la logica funcional. La orquestacion futura puede implementarse con Cloud Composer o Cloud Workflows, manteniendo dependencias explicitas entre ingesta, transformacion, publicacion y benchmark.

La eleccion de C4 Nivel 2 busca separar responsabilidades de contenedor. Los canales ciudadanos y usuarios municipales representan actores y sistemas externos; el pipeline PySpark concentra computo; Bronze, Silver y Gold son almacenes logicos con contratos distintos; BigQuery y Looker Studio son consumidores analiticos; y el bloque de gobierno GCP agrupa controles transversales. Esta mirada permite discutir escalamiento sin mezclar detalles de implementacion: si se reemplaza el notebook por un job serverless, la arquitectura funcional se mantiene; lo que cambia es el contenedor de ejecucion, la observabilidad y el modelo de costos.

3. Flujo de datos Bronze, Silver y Gold

Bronze contiene `solicitudes_raw`, `eventos_raw`, `logs_raw`, `fotografias_metadata_raw`, `clima_raw`, `cuadrillas_raw` y `satisfaccion_raw`. Silver genera `solicitudes_enriquecidas`, `eventos_normalizados`, `logs_normalizados` y dimensiones curadas. Las validaciones incluyen IDs no nulos, fechas dentro de rango, coordenadas plausibles, deduplicacion y flags como `es_vencida`, `es_critica`, `tiene_reapertura`, `tiene_fotografia`, `cumple_sla` y `posible_duplicado`. Gold publica `kpi_sla`, `backlog_operacional`, `zonas_criticas`, `satisfaccion_ciudadana`, `desempeno_cuadrillas` y `dataset_modelo_priorizacion`.

La frontera entre capas se definio para evitar acoplamiento prematuro. Bronze no corrige ni interpreta: conserva datos de entrada con tipos y particiones utiles para reprocesamiento. Silver aplica reglas de negocio y genera columnas estandarizadas, por lo que es la capa adecuada para pruebas de calidad y auditoria. Gold reduce dimensionalidad y publica entidades listas para consumo, de modo que dashboards y consultas ejecutivas no dependan de joins complejos sobre datos crudos. Este diseno tambien permite reprocesar una capa sin destruir toda la cadena: si cambia el criterio de SLA, se regenera Silver/Gold sin tener que volver a producir Bronze.

Capa	Dataset principal	Funcion
Bronze	solicitudes/eventos/logs/raw	Persistir fuentes sinteticas crudas en Parquet.
Silver	solicitudes_enriquecidas	Curar, validar, enriquecer y calcular flags operacionales.
Gold	kpi_sla/backlog/zonas/satisfaccion	Publicar indicadores para dashboard y analitica.
Modelo	dataset_modelo_priorizacion	Preparar variables disponibles al ingreso sin data leakage.

4. Evidencia ejecutada y resultados medibles

La evidencia adjunta (`run_summary.json`) registra `execution_status=success` en `SCALE_MODE=full` con Spark 4.0.3. La corrida proceso 12.429.220 filas Bronze, incluyendo 10.800.000 eventos, 1.500.000 logs y 50.000 solicitudes. El tiempo total fue 2.498,967 segundos. La calidad reporto cero IDs criticos nulos y cero coordenadas fuera de rango.

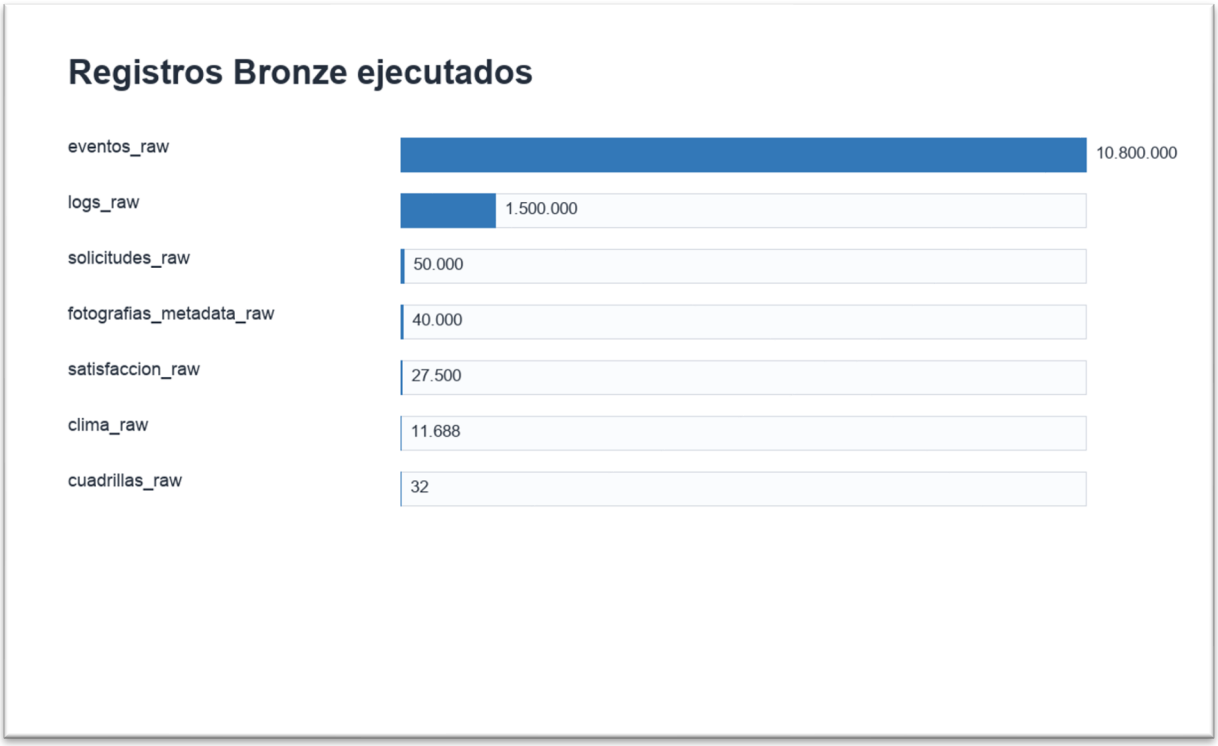


Figura 2. Volumen Bronze de la corrida adjunta.

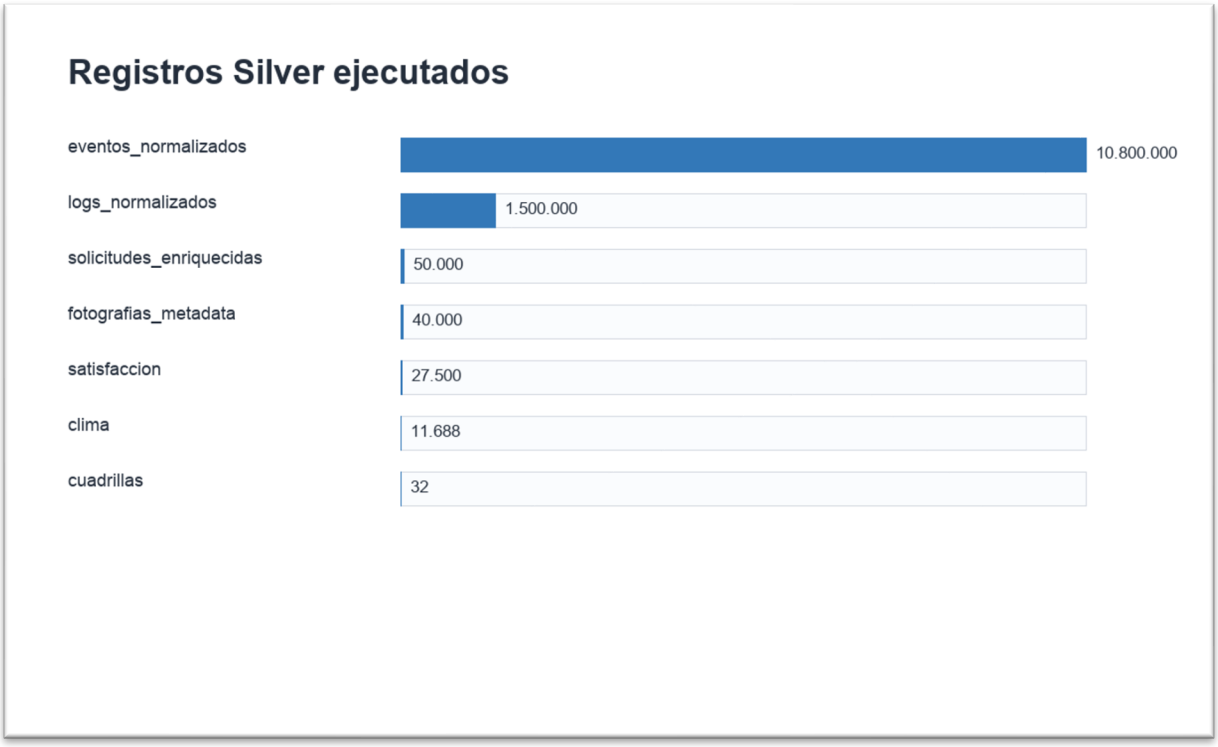


Figura 3. Volumen Silver de la corrida adjunta.



Figura 4. Volumen Gold de la corrida adjunta.

La trazabilidad por etapa (`execution_steps.json`) muestra que todas las etapas terminaron en success. La etapa mas costosa fue el benchmark de eventos, seguida por escrituras Gold/Silver especificas. Esto confirma que la instrumentacion permite ubicar cuellos de botella reales en escala full y entrega una base concreta para decidir donde escalar recursos o migrar a GCP.

La evidencia debe leerse como prueba funcional y de escala: el pipeline completo alcanzo el perfil full, pero aun asi requiere observabilidad mas robusta si se quiere operar periodicamente. El valor de `execution_steps.json` es operacional: registra inicio, fin, duracion y estado por etapa, lo que permite construir una bitacora comparable entre small, medium y full. En una ejecucion administrada, esta misma informacion deberia complementarse con logs Spark, metricas de executor, bytes leidos/escritos y alertas por etapa. Esa extension permitiria responder preguntas tecnicas concretas: si el limite aparece en shuffle, escritura Parquet, memoria de driver, cantidad de archivos pequenos o lectura desde almacenamiento remoto.

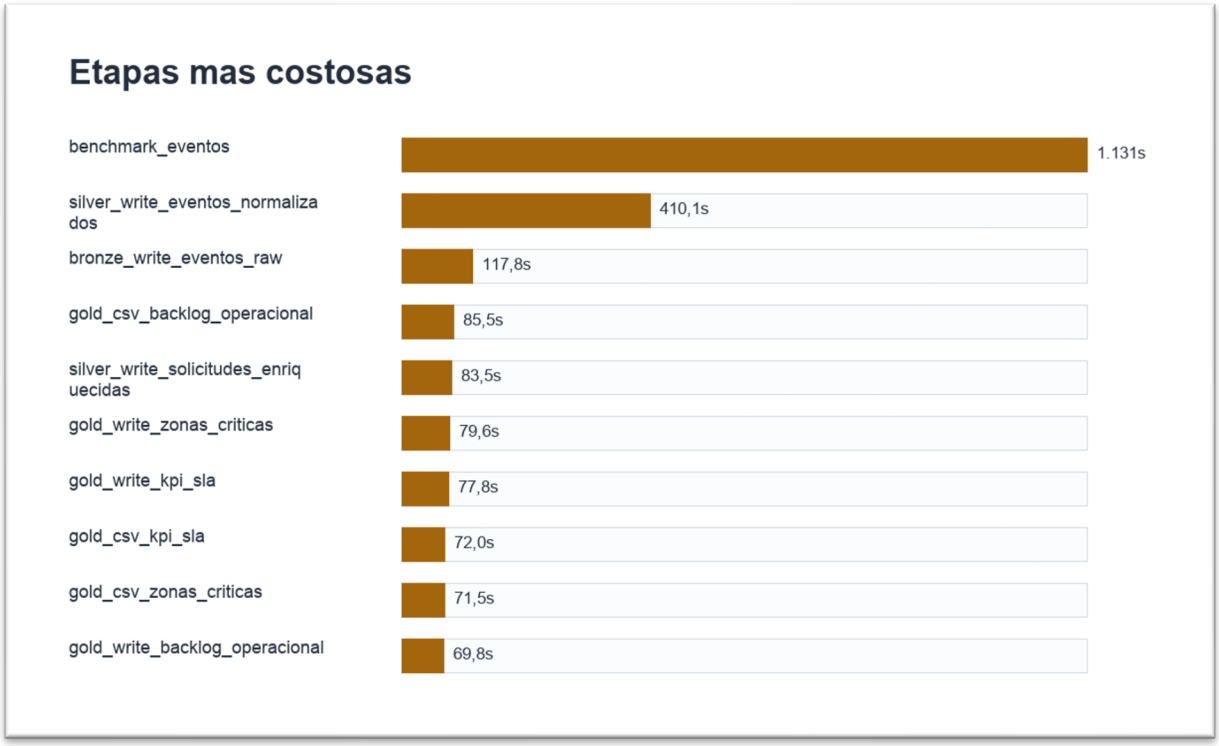


Figura 5. Etapas de mayor duracion.

5. Optimizacion, benchmark y lectura critica

El benchmark compara lectura/agregacion sobre Parquet base sin particionamiento versus Parquet particionado por `anio`, `mes`, `zona_gestion` y `tipo_evento`. En la evidencia full, el experimento optimizado fue mas lento: la lectura/agregacion paso de 2,839 s a 4,031 s, con mejora porcentual de -41,98%. Este resultado no invalida conceptualmente el particionamiento; demuestra que, para este filtro y esta materializacion, el costo de listar y abrir multiples particiones supero el beneficio de pruning. La decision correcta es mantener el benchmark y reportar el resultado negativo como evidencia tecnica, no ajustar el experimento hasta obtener una mejora artificial.

La lectura critica es relevante para un Data Engineer porque una optimizacion solo es real si mejora el patron de acceso esperado. Particionar por muchas columnas puede crear demasiados archivos pequenos, aumentar operaciones de listado y penalizar escenarios donde se consulta casi todo el dataset. En el caso municipal, la hipotesis razonable es que analistas filtren por periodos, zona o tipo de evento; por tanto, el particionamiento sigue siendo defendible como estrategia candidata. Sin embargo, antes de llevarlo a produccion debe probarse con filtros representativos, comparar tamanos de archivo, revisar skew por zona/tipo y evaluar alternativas como particionar por fecha y usar clustering o Z-order en tecnologias lakehouse mas avanzadas.

“El benchmark full mostró que el particionamiento elegido no mejoró esta consulta selectiva específica; fue 41,98% más lento. Eso no invalida el diseño, sino que evidencia un trade-off real: una partición mal alineada o con archivos pequeños puede degradar rendimiento. La recomendación es revalidar particiones en GCP con distintos patrones de consulta y compactación de archivos.”

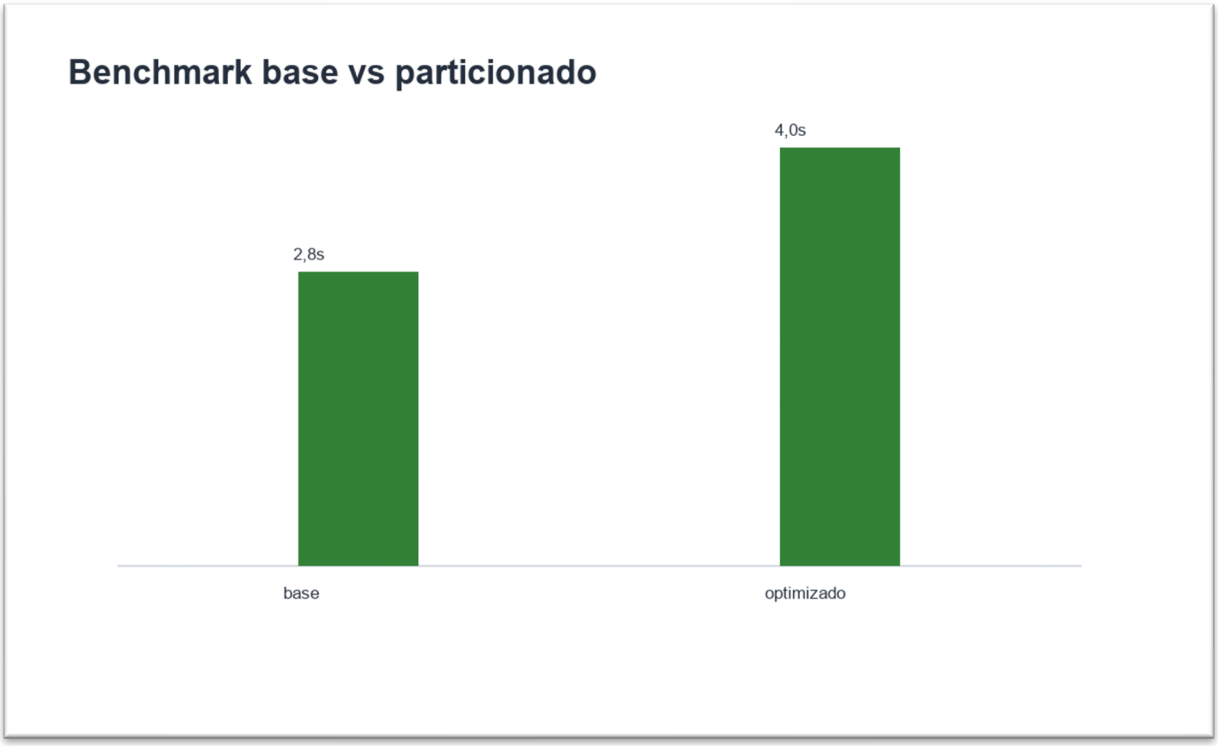


Figura 6. Benchmark de lectura/agregacion.

Experimento	Registros	Tiempo (s)	Mejora %
base	10.800.000	2,839	0,00
optimizado	10.800.000	4,031	-41,98

6. Cumplimiento full y limitaciones actuales

El enunciado recomienda al menos 10 millones de filas o 5 GB en Bronze para que las decisiones de Spark tengan impacto real. La evidencia actual cumple ese criterio: el perfil full proceso solicitudes, eventos, logs y tablas complementarias hasta alcanzar mas de 12 millones de filas Bronze. El entorno reportado tenia 12,67 GB de RAM total, 8,91 GB libres al inicio y 87,57 GB de disco libre. La corrida finalizo en 41,65 minutos, con 3,55 GB libres al cierre y 79,99 GB de disco disponible.

Componente	Full requerido	Evidencia adjunta	Estado
Solicitudes	50.000	50.000	Cumple
Eventos	10.800.000	10.800.000	Cumple
Logs	1.500.000	1.500.000	Cumple
Bronze total	> 10.000.000	12.429.220	Cumple

Las limitaciones actuales ya no son de cumplimiento full, sino de operacion productiva: la ejecucion sigue dependiendo de un entorno interactivo, la observabilidad se basa en JSON locales y los datos Parquet pesados no deben almacenarse ni sincronizarse directamente en Google Drive. La contribucion de Fase 3 es dejar el sistema documentado con evidencia full reproducible, control de errores, rutas seguras de Colab y una ruta cloud para repetir la escala con operacion administrada.

La principal recomendacion es no convertir este exito puntual en una operacion manual recurrente. En entornos locales o Colab, el riesgo sigue estando en memoria del driver, disco temporal, cantidad de archivos generados, limites de Drive y tiempo maximo de sesion. La alternativa profesional es conservar el perfil full, versionar evidencia, estimar recursos y mover la ejecucion a un motor administrado que permita configurar workers, memoria, disco local, red y observabilidad.

7. Manual operativo resumido

- Instalar dependencias con `pip install -r requirements.txt` y configurar `PYTHONPATH=src`.
- Ejecutar prueba rapida con `--scale-mode small` para validar dependencias.
- Para full, ejecutar en `/content/fase2_gestion_territorial_el_monte`, usar `/content/spark_tmp`, `/content/spark_warehouse` y `/content/spark_checkpoints`, y no escribir Parquet masivo en Drive.
- Revisar `evidence/run_summary.json`, `execution_steps.json` y `optimization_metrics.json` al finalizar.
- Si `execution_status=failed`, reportar `failed_step` y `error_message`; no bajar escala para declarar exito.
- Copiar a Drive solo ZIP final y evidencia, no la carpeta `data/` completa.

8. Gobierno, seguridad, linaje y FinOps

La solucion usa datos sinteticos, identificadores ciudadanos hasheados y rutas de fotografias simuladas, por lo que no expone datos personales reales. El gobierno se apoya en separacion por capas, linaje manual, controles de calidad y un modelo RBAC simulado: administrador tecnico, analista de datos, supervisor municipal, funcionario operativo y autoridad. En GCP, estos controles deben evolucionar a IAM por service accounts, cifrado con KMS, Cloud Audit Logs, Cloud Logging y politicas de retencion de Cloud Storage.

Desde FinOps, el costo debe estimarse con variables editables: GB almacenados en Parquet, duracion promedio de jobs Spark, numero de ejecuciones mensuales, GB leidos por consulta BigQuery y frecuencia del dashboard. La recomendacion es medir costo por corrida y costo por KPI publicado, evitando recalcular Gold completo cuando basten particiones incrementales.

El linaje propuesto registra origen, transformacion y salida por dataset. Para una municipalidad, esta trazabilidad tiene impacto practico: si una autoridad cuestiona un indicador de backlog, el equipo tecnico debe reconstruir que reglas generaron el KPI, que datos fueron excluidos por calidad y que version del pipeline produjo el resultado. En una evolucion GCP, ese linaje puede integrarse con Data Catalog o Dataplex, etiquetas de sensibilidad, owners por dataset y politicas de retencion. La combinacion de gobierno y FinOps evita dos riesgos frecuentes: abrir demasiado acceso por urgencia operacional y dejar pipelines costosos corriendo sin medicion de valor.

9. Propuesta de evolucion GCP

Capa	Servicio GCP	Rol
Storage	Cloud Storage	Data lake Bronze/Silver/Gold en Parquet con lifecycle y prefijos por capa.
Procesamiento	Dataproc Serverless	Ejecucion Spark full con recursos configurables, logs y aislacion.
Analitica	BigQuery	Tablas externas o carga de Gold agregado para consultas SQL.
Visualizacion	Looker Studio	Dashboard municipal ejecutivo, territorial, SLA/backlog y satisfaccion.
Gobierno	IAM, KMS, Audit Logs	Minimo privilegio, cifrado, auditoria y trazabilidad.
ML futuro	Vertex AI	Modelo de priorizacion con validacion humana y monitoreo.

La migracion sugerida puede ejecutarse por etapas. Primero, mover los datos y outputs a Cloud Storage manteniendo la misma estructura de carpetas medallion. Segundo, ejecutar el job PySpark en Dataproc Serverless con parametros de escala y logging centralizado. Tercero, publicar Gold en BigQuery, idealmente con tablas externas para exploracion y tablas materializadas para indicadores de alto uso. Cuarto, conectar Looker Studio a vistas controladas y no directamente a Bronze/Silver. Finalmente, incorporar monitoreo de calidad y costos por ejecucion. Esta ruta minimiza cambios funcionales y concentra la evolucion en infraestructura, gobierno y operacion.

10. Conclusiones

La Fase 3 consolida un proyecto coherente entre arquitectura, implementacion y documentacion. El pipeline demuestra flujo end-to-end en full, resultados Gold, controles de calidad, benchmark y evidencia por etapa. La evidencia disponible supera el umbral Big Data y debe presentarse como resultado real de ejecucion, junto con una lectura critica: el particionamiento medido no mejoro el tiempo del filtro evaluado. El siguiente paso tecnico es correr el mismo repositorio en Dataproc/Spark Serverless, persistir Parquet en Cloud Storage, publicar Gold en BigQuery y validar el benchmark con mas patrones de consulta.

En terminos academicos, el entregable cumple el objetivo de Fase 3 porque une arquitectura final, decisiones justificadas, manual operativo, evidencia full y ADRs. En terminos profesionales, deja una base defendible para conversar con un equipo de plataforma: que recursos se usaron, que riesgos se conocen, que controles ya existen y que partes conviene administrar en GCP. La recomendacion final es tratar la corrida full como prueba de carga lograda y la migracion cloud como paso natural para repetibilidad, gobierno y monitoreo.

Declaración de uso de IA generativa.

El equipo utilizó herramientas de IA generativa, incluyendo ChatGPT y Codex, como apoyo para estructurar documentación técnica, revisar redacción, revisar propuestas de ADRs, orientar ajustes del notebook y mejorar la organización del repositorio. El diseño, validación, ejecución del pipeline, revisión de resultados y responsabilidad final del contenido corresponden al equipo.

Anexo A - Architecture Decision Records

ADR-001: Arquitectura lakehouse medallion

Contexto

El caso municipal combina solicitudes ciudadanas, eventos operacionales, logs, clima, cuadrillas, fotografias metadata y satisfaccion. La solucion requiere separar datos crudos, datos curados y datasets de consumo analitico.

Decision

Adoptar una arquitectura lakehouse medallion con capas Bronze, Silver y Gold, persistidas en Parquet.

Alternativas consideradas

- Una unica tabla analitica: simple, pero poco trazable y fragil ante cambios.
- Base relacional tradicional: conocida, pero menos adecuada para datos semiestructurados y volumen operacional.
- Data lake sin capas: flexible, pero con mayor riesgo de desorden y baja gobernabilidad.

Consecuencias

La separacion mejora trazabilidad, calidad y gobierno. El costo es mayor complejidad operacional y necesidad de documentar contratos entre capas.

ADR-002: PySpark y Parquet para procesamiento batch

Contexto

El enunciado exige demostrar Big Data con al menos 10 millones de filas o 5 GB en Bronze. El pipeline debe ser reproducible en Colab/GCP y no depender de pandas.

Decision

Usar PySpark como motor distribuido y Parquet con compresion Snappy como formato de almacenamiento.

Alternativas consideradas

- pandas: descartado porque no representa una solucion Big Data.
- CSV: util para intercambio, pero ineficiente para lectura selectiva y tipos.
- Delta/Iceberg: valiosos para produccion, pero agregan dependencias no necesarias para el alcance academico.

Consecuencias

PySpark permite generacion distribuida con `spark.range`, transformaciones escalables y particionamiento. Parquet reduce I/O y habilita analitica posterior. Requiere controlar particiones, memoria y escritura temporal.

ADR-003: Ejecucion reproducible en Colab con ruta a GCP administrado

Contexto

El proyecto debia demostrar escala Big Data sin reducir artificialmente el perfil full. Despues de varios intentos, la ejecucion full pudo completarse con evidencia controlada.

Decision

Mantener el notebook preparado para full, ejecutar el perfil completo sin reducir filas y conservar evidencia JSON/CSV de la corrida. La ruta GCP sigue recomendada para operacion repetible y gobernada.

Alternativas consideradas

- Reducir full para hacerlo pasar: descartado por falta de honestidad tecnica.
- Forzar ejecucion local: descartado por memoria/disco limitados.
- Migrar a Dataproc inmediatamente: recomendado para produccion, pero no fue indispensable para obtener la evidencia academica full.

Consecuencias

La evidencia full demuestra cumplimiento de escala y reproducibilidad end-to-end. La recomendacion de GCP queda como evolucion operacional, no como excusa por falta de evidencia.

ADR-004: Benchmark de particionamiento y lectura critica

Contexto

La Fase 2 exige comparar una version base sin particionamiento contra una version optimizada. La evidencia full adjunta muestra que la lectura particionada fue mas lenta para el filtro selectivo medido.

Decision

Mantener el benchmark base versus particionado por anio, mes, zona_gestion y tipo_evento, reportando el resultado negativo como evidencia tecnica.

Alternativas consideradas

- Omitir benchmark: incumple el enunciado.
- Ajustar filtros hasta obtener mejora: riesgoso si no representa uso real.
- Reportar solo escritura: insuficiente para analizar consumo analitico.

Consecuencias

El benchmark ensena que particionar tiene costo y que su beneficio depende de volumen, selectividad, numero de archivos y patron de consulta. En este experimento full no fue favorable; debe revalidarse con otros filtros y en GCP.

ADR-005: Gobierno, seguridad y datos sinteticos

Contexto

El caso simula informacion ciudadana municipal. Aunque los datos son sinteticos, el diseno debe considerar privacidad, roles y trazabilidad.

Decision

Usar datos sinteticos, identificadores hasheados, linaje manual, separacion por capas y RBAC simulado por rol.

Alternativas consideradas

- Datos reales municipales: descartados por privacidad y autorizacion.
- Sin controles de acceso: no cumple buenas practicas ni el enunciado.
- Anonimizacion posterior: menos segura que diseñar desde el origen con datos sinteticos.

Consecuencias

La solucion es segura para entrega academica y deja una ruta clara hacia IAM, KMS, Cloud Logging y auditoria en GCP.